Dissertation Weekly Progress Report
Week 3

Date of Last Meeting: 16 October 2026
Date of Next Meeting: 23 October 2026
Current Focus Area: Primordial Power Spectra, CAMB Implementation, Bayesian Inference

---

This week has been rather busy, so I knew I would not be able to do my usual amount of reading/writing. Because of this, I decided that I would spend the time I have this week learning the CAMB [1] Python package. I arrive at (I believe) novel measurements/constraints of the spectral index n_s and the amplitude A_s from HSC galaxy clustering data. These do not offer an improvement on the current constraints, but they are in agreement and gave me good practice in using CAMB as well as setting up another MCMC implementation. Throughout the following work, I make use of my own Python backend APEX (Angular Power spectrum EXplorer) just to make some things more readable or efficient. I hope that each use of APEX is understandable to the reader. I hope to write full documentation for this code when time permits.

1 Background

For now I will state some essential results without the 'story' behind them. When applicable, I will ensure to fully flesh these ideas out.

From Baumann [2] and adapted from lecture notes by [3] we have the following scaling for the matter power spectrum:

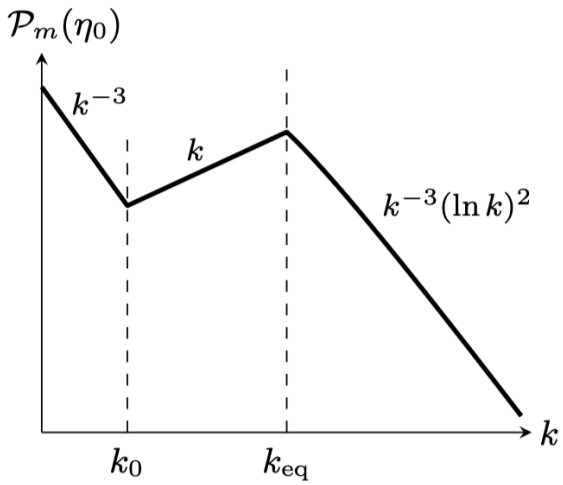


Figure 1: The final scalings for the matter power spectrum as derived in [2]. Scales $k < k_0$ are superhorizon modes today and thus are unobservable.

The other result we will need is the *Primordial Power Spectrum*. This is a statistical measure of variance of the primordial curvature perturbations at spatial scales given by k. It has the form

$$\mathcal{P}_{\mathcal{R}}(k) = A_s \left(\frac{k}{k_*}\right)^{n_s - 1}$$

where A_s is the scalar amplitude of the perturbations, n_s is the spectral index, and k_* is the pivot scale (a reference scale). For our purposes we use $k_* = 0.05$ Mpc^{-1}

2 Preliminary CAMB Work

I first compute present-time matter power spectrua, varying the primordial power spectrum variables to show how they effect the late time structure.

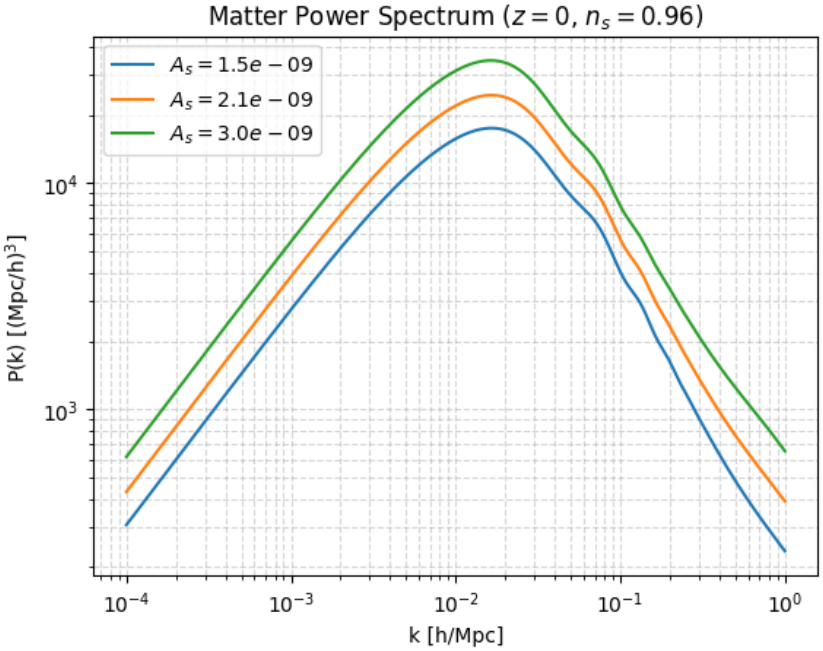


Figure 2:

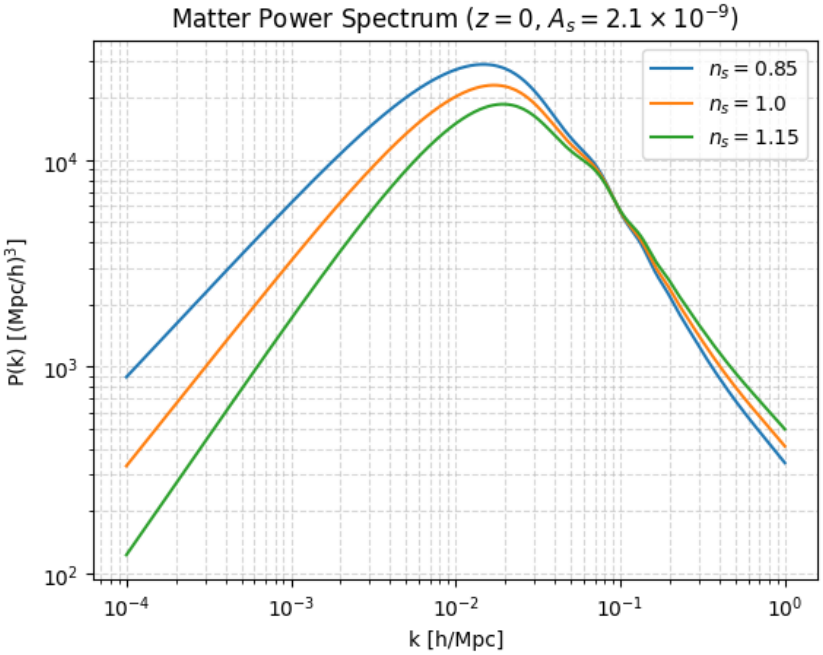


Figure 3:

Now I use the *Planck*[4] values for the variables to compute the matter power spectrum we expect to see from these values:

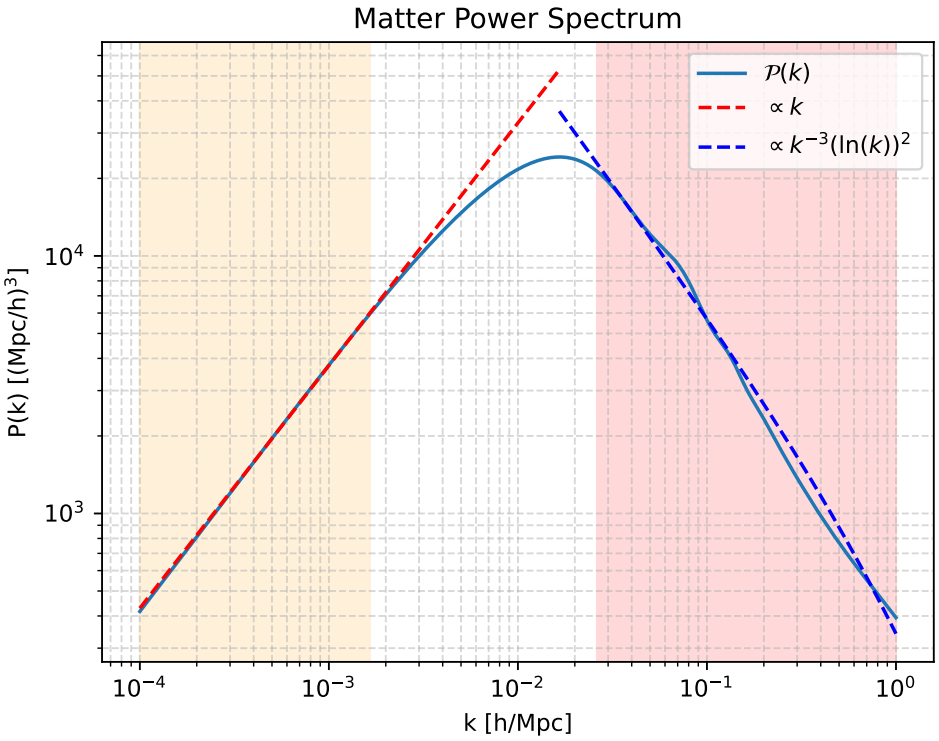


Figure 4:

This figure also includes best fit to the scalings shown in 1. The shaded regions show the data points used in the fitting of each regime.

The code for generating such a power spectrum looks like:

```
def compute_pk(ns, As):

    pars = camb.CAMBparams()
    pars.set_cosmology(H0=67.5, ombh2=0.022, omch2=0.122)

    pars.InitPower.set_params(As=As, ns=ns)

    pars.set_matter_power(redshifts=[0.0], kmax=2.0)
    pars.NonLinear = camb.model.NonLinear_both
    pars.SourceTerms.limber_windows = True
    pars.SourceTerms.counts_redshift = True
    pars.SourceTerms.counts_lensing = False

    results = camb.get_results(pars)

    kh, _, pk = results.get_matter_power_spectrum(minkh=1e-4, maxkh=1.0, npoints
    ↪  = 2000)

    return kh, pk
```

3 Measurements of n_s and A_s from HSC Data

3.1 Angular Power Spectra

In CAMB, to compute an angular power spectrum C_ℓ one must define the windows being used (that is, which tracers are being correlated by the power spectrum). For this we use an auto-correlation of galaxy clustering, with the resulting power spectrum denoted C_ℓ^{gg}. This can be implemented as

```
camb.sources.SplinedSourceWindow(
            source_type = 'counts',
            bias = b_g,
            z = z,
            W = nz,
            )
```

where `z` is the redshift value array, and `nz` is the distribution function. Using a `SplinedSourceWindow` requires these to be given explicitly. For this reason we will use the $n(z)$ distribution for the Hyper-Suprime Camera galaxy clustering (which I have to hand given my research in to the use of the CIB) [1]. This distribution is split into 4 bins:

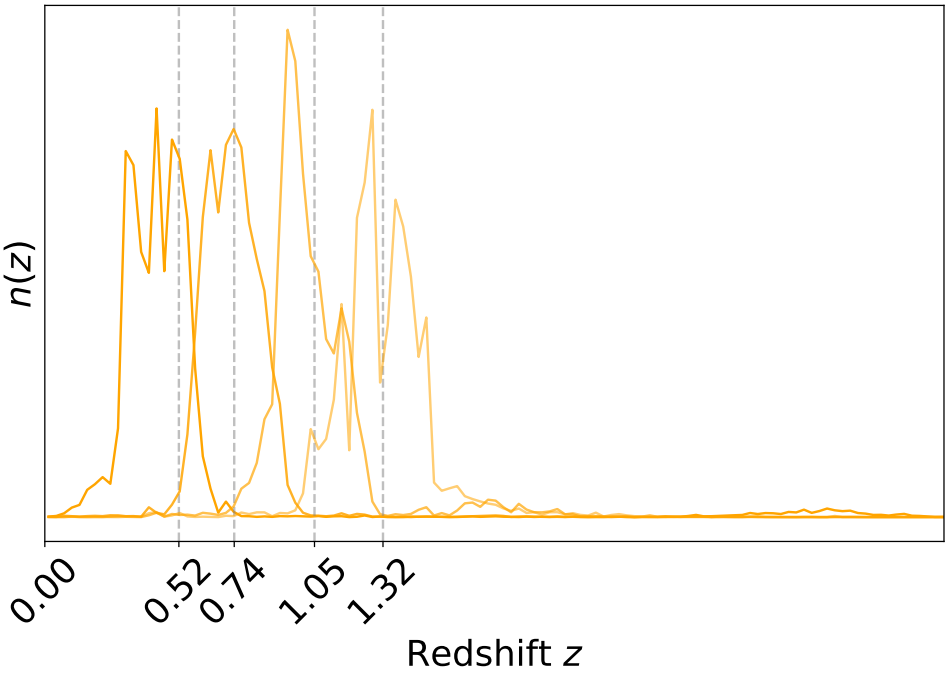


Figure 5:

To ensure that this works well with CAMB, I employ some mild pre-processing:

```
def preprocess_nz(self, sacc_file, tracer, plot=False):
        data = sacc.Sacc.load_fits(sacc_file).get_tracer(tracer)

        z = data.z
        nz = data.nz
```

[1] We are aware of certain systematic issues that may be inherent to this distribution, but for illustrative purposes here, it seems to work well.

```
    nz_smooth = gaussian_filter1d(nz, sigma=0.2)
    peak_val = np.max(nz_smooth)
    nz_smooth[nz_smooth < 1e-1 * peak_val] = 0.0

    non_zero = np.where(nz_smooth > 0)[0]
    if len(non_zero) > 0:
        idx_start = max(0, non_zero[0] - 2)
        idx_end = min(len(z), non_zero[-1] + 3)

        z_clean = z[idx_start:idx_end]
        nz_clean = nz_smooth[idx_start:idx_end]

        # Ensure exact zero at boundary endpoints
        nz_clean[0] = 0.0
        nz_clean[-1] = 0.0

    if plot:
        plt.figure(figsize=(8, 6))
        plt.plot(z, nz, label='Original $n(z)$', color='blue', alpha=0.5)
        plt.plot(z_clean, nz_clean, label='Smoothed $n(z)$', color='orange', alpha=0.8)
        plt.xlabel('Redshift $z$', fontsize=12)
        plt.ylabel('Normalized $n(z)$', fontsize=12)
        plt.title(f'Preprocessing $n(z)$ for tracer: {tracer}', fontsize=13)
        plt.legend(fontsize=10)
        plt.grid(True, which='both', linestyle=':', alpha=0.6)
        plt.show()

    return z_clean, nz_clean
```

This function is taken from a class object that I will give in full later. The effect of this pre-processing can be illustrated (for one bin) as follows:

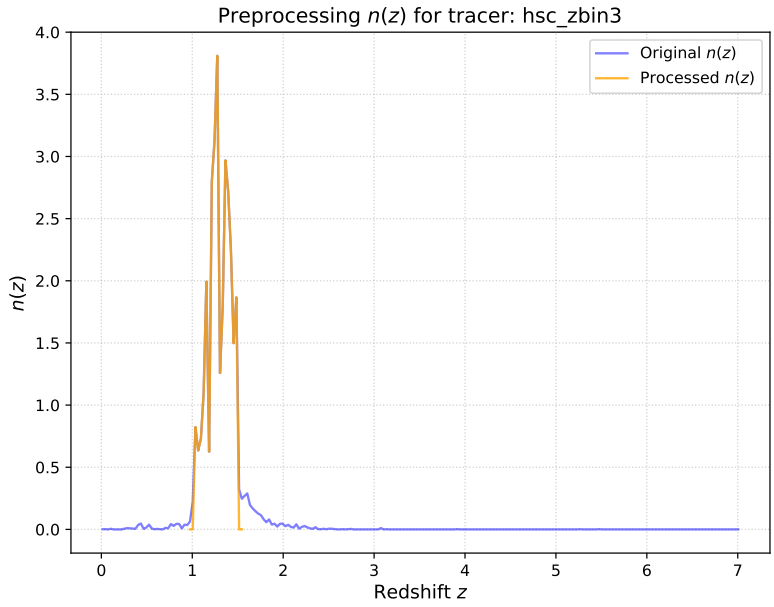


Figure 6:

Without (yet) giving the code, we can use this to compute the C_ℓ^{gg} from CAMB, based on HSC redshift distributions, and using the galaxy bias b_g obtained in my previous work on CIB tomography, for one of the redshift bins. This takes the same assumptions for the primordial power spectrum variables as in Figure 4.

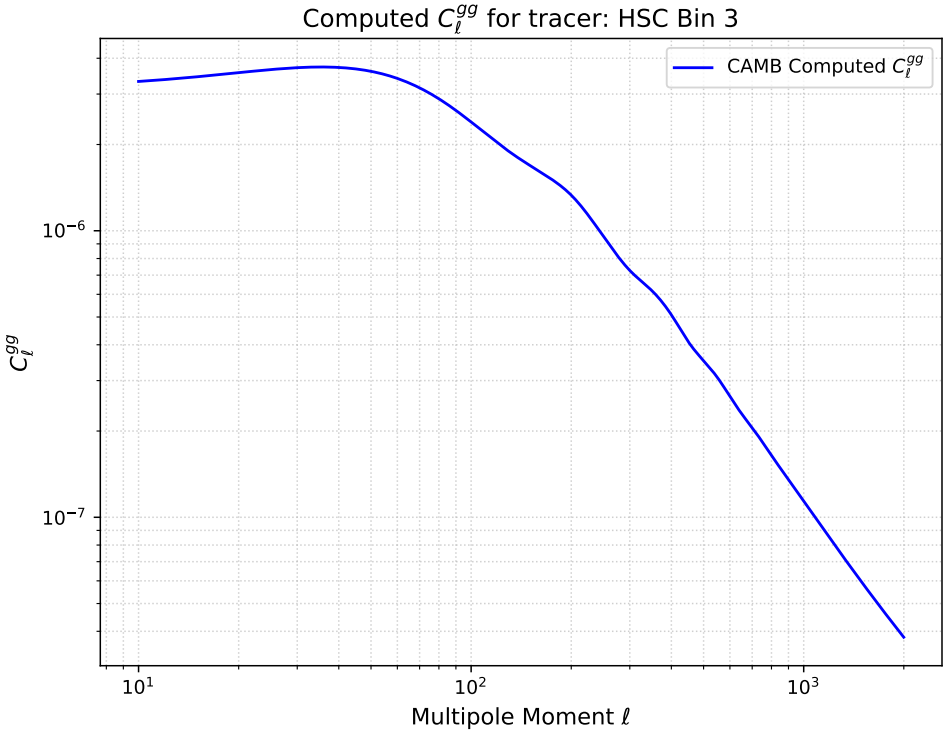


Figure 7:

3.2 Markov Chain Monte Carlo Implementation

Now we have a working (we hope) model that has its only free parameters as $\{n_s, A_s\}$. This motivates us to try to use the data we have to constrain (using this model) the values of these parameters, gaining a posterior distribution of them. We do this through the use of MCMC, using the `cobaya` Python package [5, 6].

We employ a Gaussian log-likelihood of the form

$$\ln \mathcal{L} = -\frac{1}{2}\Delta^{\mathsf{T}}\Sigma^{-1}\Delta$$

where:

- $\mathbf{d}$ is the data vector, which is constructed to contain the C_ℓ^{gg} measurements from all 4HSC bins
- Δ is the residual vector. That is, the difference between the modelled C_ℓ^{gg} and $\mathbf{d}$
- Σ is the covariance matrix of the data used.

We mask the observed C_ℓ^{gg} to only include the data in the range $100 < \ell < 1000$ when fitting, to reduce contamination at very large or very small scales.

Thus we get the values (and errors):

$$\boxed{n_s = 0.95427 \pm 0.02002 \qquad A_s = (1.72379 \pm 0.03431) \times 10^{-9}}$$

The value for n_s is in agreement with [4], but the value for A_s is not. I am not too worried by this as this was only as a practice using CAMB for modelling.

The MCMC was ran with a convergence $R-1$ value of only 0.1, this was done to spped up computation, but would not suffice if we were preparing this for a paper. The resultant corner plot is given below:

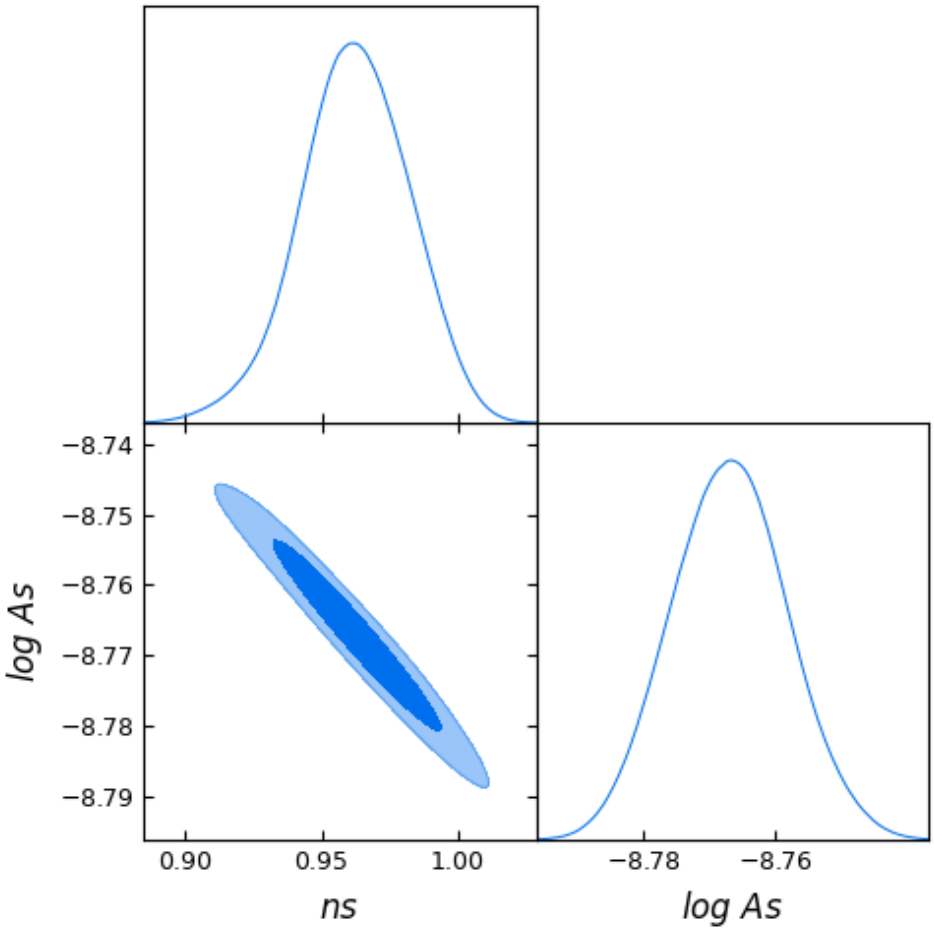


Figure 8:

The best fit values lead to the fit shown here:

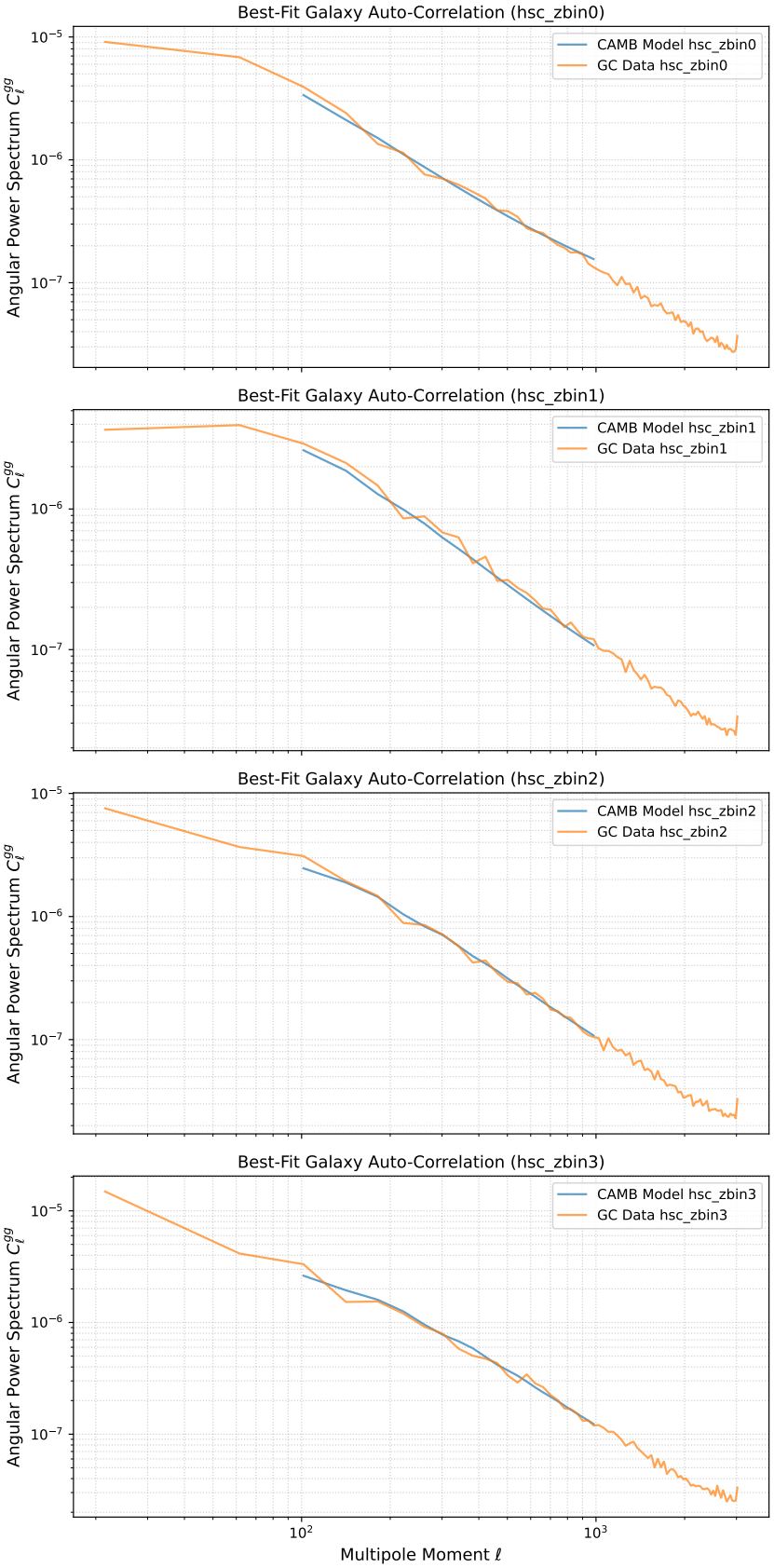


Figure 9:

A similar process can be done on DELS galaxy clustering data giving the fit:

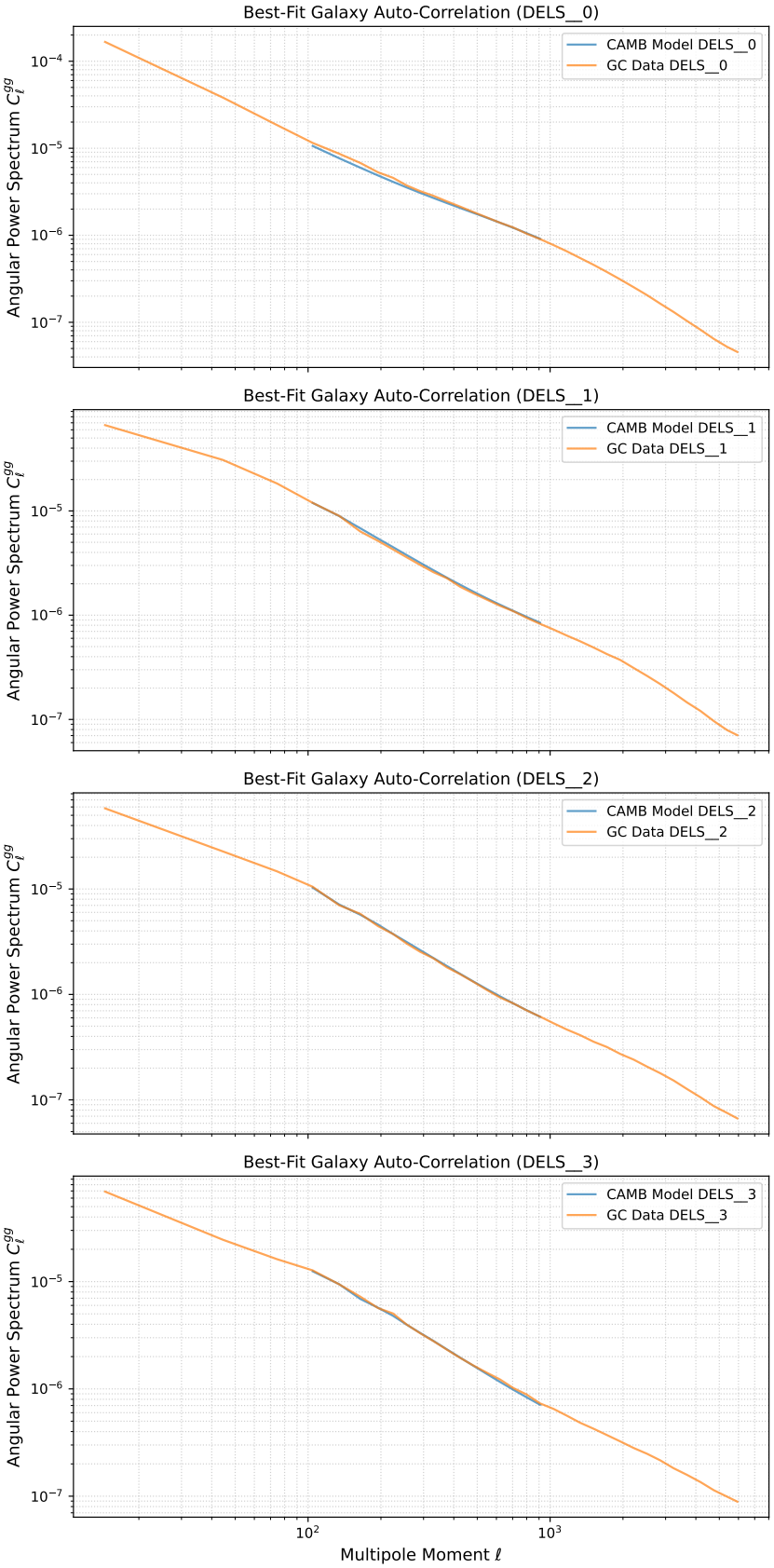

Best-Fit Galaxy Auto-Correlation (DELS__0)
CAMB Model DELS__0
GC Data DELS__0
Best-Fit Galaxy Auto-Correlation (DELS__1)
CAMB Model DELS__1
GC Data DELS__1
Best-Fit Galaxy Auto-Correlation (DELS__2)
CAMB Model DELS__2
GC Data DELS__2
Best-Fit Galaxy Auto-Correlation (DELS__3)
CAMB Model DELS__3
GC Data DELS__3
Angular Power Spectrum C_ℓ^{gg}
10^{-4}
10^{-5}
10^{-6}
10^{-7}
10^{2}
10^{3}
Multipole Moment ℓ

Figure 10:

3.3 The Code

I will provide the main non-trivial sections of code for this implementation:

```
class CAMB_gc:

    def __init__(self, tracers= None, biases = None, sacc_workspace=None,
    ↪  l_min=100, l_max=2000):
        self.biases = biases
        self.l_min = l_min
        self.l_max = l_max

        self.data = {}
        self.icovariances = {}
        self.gc_masks = {}
        self.redshift_info = {}
        i=0

        self.data_ells = []
        self.data_vector = []
        masks = []

        if tracers is not None:
            self.tracers = tracers
            for tracer in self.tracers:

                gc_ells =
                ↪  np.array(sacc_workspace.get_c_ells(tracer_combinations=[(tracer,
                ↪  tracer)]))[0][0]
                gc_c_ells =
                ↪  np.array(sacc_workspace.get_c_ells(tracer_combinations=[(tracer,
                ↪  tracer)]))[1][0]

                self.data[tracer] =
                ↪  [np.array(sacc_workspace.get_c_ells(tracer_combinations=[(tracer,
                ↪  tracer)])), biases[i]]

                mask_gc = (gc_ells >= l_min) & (gc_ells <= l_max)
                self.gc_masks[tracer] = mask_gc

                self.data_ells.append(gc_ells[mask_gc])
                self.data_vector.append(gc_c_ells[mask_gc])

                masks.append(mask_gc)
                i+=1

                z, nz = self.preprocess_nz(sacc_workspace.sacc_file, tracer,
                ↪  False)
                self.redshift_info[tracer] = (z, nz)

            self.data_vector = np.concatenate(self.data_vector)
```

```
        self.icovariance =
        ↪  np.linalg.inv(sacc_workspace.cut_covariance_matrix(datatype='cl_00',
        ↪  tracer_combos=[(tracer, tracer) for tracer in self.tracers],
        ↪  masks=masks))


    self.pars = camb.CAMBparams()
    self.pars.set_cosmology(H0=67.5, ombh2=0.022, omch2=0.122)

def preprocess_nz(self, sacc_file, tracer, plot=False):
    data = sacc.Sacc.load_fits(sacc_file).get_tracer(tracer)

    z = data.z
    nz = data.nz

    nz_smooth = gaussian_filter1d(nz, sigma=0.2)
    peak_val = np.max(nz_smooth)
    nz_smooth[nz_smooth < 1e-1 * peak_val] = 0.0

    non_zero = np.where(nz_smooth > 0)[0]
    if len(non_zero) > 0:
        idx_start = max(0, non_zero[0] - 2)
        idx_end = min(len(z), non_zero[-1] + 3)

        z_clean = z[idx_start:idx_end]
        nz_clean = nz_smooth[idx_start:idx_end]

        # Ensure exact zero at boundary endpoints
        nz_clean[0] = 0.0
        nz_clean[-1] = 0.0

    if plot:
        plt.figure(figsize=(8, 6))
        plt.plot(z, nz, label='Original $n(z)$', color='blue', alpha=0.5)
        plt.plot(z_clean, nz_clean, label='Smoothed $n(z)$', color='orange',
        ↪  alpha=0.8)
        plt.xlabel('Redshift $z$', fontsize=12)
        plt.ylabel('Normalized $n(z)$', fontsize=12)
        plt.title(f'Preprocessing $n(z)$ for tracer: {tracer}', fontsize=13)
        plt.legend(fontsize=10)
        plt.grid(True, which='both', linestyle=':', alpha=0.6)
        plt.show()

    return z_clean, nz_clean


def compute_full_c_ell(self, ns, As, logged = True):
    windows = []

    if logged:
        As = np.power(10, As)
```

```
        for tracer, b_g in zip(self.tracers, self.biases):

            z, nz = self.redshift_info[tracer]

            self.pars.InitPower.set_params(ns=ns, As=As, pivot_scalar=0.05)

            galaxy_clustering_window = camb.sources.SplinedSourceWindow(
                source_type = 'counts',
                bias = b_g,
                z = z,
                W = nz,
                )

            windows.append(galaxy_clustering_window)

        self.pars.SourceWindows = windows

        # Add complexity as necessary
        self.pars.SourceTerms.limber_windows = True
        self.pars.NonLinear = camb.model.NonLinear_both
        self.pars.SourceTerms.counts_redshift = True
        self.pars.SourceTerms.counts_lensing = False

        self.pars.set_for_lmax(self.l_max, lens_potential_accuracy=1)

        results = camb.get_results(self.pars)
        cls = results.get_source_cls_dict(raw_cl=True)
        results = camb.get_results(self.pars)
        cls_dict = results.get_source_cls_dict(raw_cl=True)

        model_vectors = []
        for i, tracer in enumerate(self.tracers):
            # Window auto-correlations are key-indexed as W1xW1, W2xW2, etc.
            cl_model = cls_dict[f'W{i+1}xW{i+1}']
            ell_camb = np.arange(len(cl_model))

            # Interpolate to data multipoles
            cl_interp = np.interp(self.data_ells[i], ell_camb, cl_model)
            model_vectors.append(cl_interp)

        return np.concatenate(model_vectors)

    def compute_single_c_ell(self, ns, As, tracer):
        bias = self.biases[self.tracers.index(tracer)]
        z, nz = self.redshift_info[tracer]

        self.pars.InitPower.set_params(ns=ns, As=As, pivot_scalar=0.05)

        galaxy_clustering_window = camb.sources.SplinedSourceWindow(
            source_type = 'counts',
            bias = bias,
            z = z,
```

```
            W = nz,
            )

        self.pars.SourceWindows = [galaxy_clustering_window]

        # Add complexity as necessary
        self.pars.SourceTerms.limber_windows = True
        self.pars.NonLinear = camb.model.NonLinear_both
        self.pars.SourceTerms.counts_redshift = True
        self.pars.SourceTerms.counts_lensing = False

        self.pars.set_for_lmax(self.l_max, lens_potential_accuracy=1)

        results = camb.get_results(self.pars)
        cls_dict = results.get_source_cls_dict(raw_cl=True)

        cl_model = cls_dict['W1xW1']
        ell_camb = np.arange(len(cl_model))

        # Interpolate to data multipoles
        cl_interp = np.interp(self.data_ells[self.tracers.index(tracer)],
        ↪  ell_camb, cl_model)

        return self.data_ells[self.tracers.index(tracer)], cl_interp

    def multi_bin_log_likelihood(self, ns, log_As):

        As = np.power(10, log_As)
        total_logL = 0.0

        # Fixed call inside multi_bin_log_likelihood:
        cl_model = self.compute_full_c_ell(
            ns, As, logged = False
        )

        # Compute log-likelihood (assuming Gaussian errors)
        diff = cl_model - self.data_vector
        icov = self.icovariance

        logL = -0.5 * np.dot(diff, np.dot(icov, diff))
        total_logL += logL

        return total_logL

    def plot_all_bins(self, ns, log_As):
        As = np.power(10, log_As)

        # Create subplots based on the number of tracers
        num_tracers = len(self.tracers)
        fig, axes = plt.subplots(num_tracers, 1, figsize=(8, 4 * num_tracers),
        ↪  sharex=True)
```

```
        # Ensure axes is iterable even if there is only 1 tracer
        if num_tracers == 1:
            axes = [axes]

        for ax, tracer in zip(axes, self.tracers):
            gc_ells = self.data[tracer][0][0][0]
            gc_c_ells = self.data[tracer][0][1][0]

            ell, cl_model = self.compute_single_c_ell(ns, As, tracer)

            # Plot directly on the specific axis object `ax`
            ax.loglog(ell, cl_model, label=f'CAMB Model {tracer}', alpha=0.7)
            ax.loglog(gc_ells, gc_c_ells, label=f'GC Data {tracer}', alpha=0.7)

            ax.set_ylabel(r'Angular Power Spectrum $C_\ell^{gg}$', fontsize=12)
            ax.set_title(f'Best-Fit Galaxy Auto-Correlation ({tracer})',
            ↪  fontsize=12)
            ax.grid(True, which='both', linestyle=':', alpha=0.6)
            ax.legend(fontsize=10)

        # Set x-label on the bottom plot
        axes[-1].set_xlabel(r'Multipole Moment $\ell$', fontsize=12)

        plt.tight_layout()

        plt.show()
```

And then to run the MCMC, using my APEX implementation:

```
import cobaya as cb

camb_hsc = CAMB_gc(tracers=['hsc_zbin0', 'hsc_zbin1', 'hsc_zbin2', 'hsc_zbin3'],
↪  biases=[0.937592,  1.110956,  1.281957,  1.633978], sacc_workspace=s,
↪  l_min=100, l_max=1000)

mcmc = ap.MCMCWorkspace(model = camb_hsc, likelihood_function =
↪  'multi_bin_log_likelihood', sacc_file = sc_file)

params = ['ns', 'log_As']

mcmc.set_param_priors(
    params=params,
    priors=[
        (0.8, 1.2),       # ns
        (-10.0, -7.0)     # log10_As
    ]
)

# 2. Reference value near Planck baseline: log10(2.1e-9) ~ -8.678
mcmc.set_param_references(
    params=params,
    references=[
        0.95427, -8.763515
```

```
    ]
)

# 3. One proposal step per parameter
mcmc.set_param_proposals(
    params=params,
    proposals=[
        0.01,            # ns proposal width
        0.02             # log10_As proposal width
    ]
)

mcmc.MCMC_config(
    params,
    sampler_info={'mcmc': {"max_tries": 10000, "proposal_scale": 0.67,
    ↪  "Rminus1_stop": 0.1}}
)

# For best-fit
mcmc.minimize_run()

# For full MCMC sampling, run serially (MPI option also available):
info, sampler = mcmc.serial_run()
```

References

[1] Antony Lewis. "CAMB 2: cosmological power spectra for high-precision surveys". In: *Journal of Cosmology and Astroparticle Physics* 2026.10 (Oct. 2026), p. 013. ISSN: 1475-7516. DOI: 10.1088/1475-7516/2026/10/013. URL: http://dx.doi.org/10.1088/1475-7516/2026/10/013.

[2] Daniel Baumann. *Cosmology*. Cambridge Aspire website. ISBN: 9781108937092. June 30, 2022. DOI: 10.1017/9781108937092. URL: https://www.cambridge.org/highereducation/books/cosmology/53783DD7B3CB15E2E37ADFBC0C1B930F (visited on 10/11/2026).

[3] M. Amin. *Cosmology (Lecture Notes)*.

[4] N. and Aghanim et al. "Planck 2018 results: VI. Cosmological parameters". In: *Astronomy and Astrophysics* 641 (2020), A6. ISSN: 1432-0746. DOI: 10.1051/0004-6361/201833910. URL: http://dx.doi.org/10.1051/0004-6361/201833910.

[5] Jesús Torrado and Antony Lewis. "Cobaya: code for Bayesian analysis of hierarchical physical models". In: *Journal of Cosmology and Astroparticle Physics* 2021.05 (May 2021), p. 057. ISSN: 1475-7516. DOI: 10.1088/1475-7516/2021/05/057. URL: http://dx.doi.org/10.1088/1475-7516/2021/05/057.

[6] Antony Lewis. "Efficient sampling of fast and slow cosmological parameters". In: *Physical Review D* 87.10 (May 2013). ISSN: 1550-2368. DOI: 10.1103/physrevd.87.103529. URL: http://dx.doi.org/10.1103/PhysRevD.87.103529.